

Rovauto Backend Flow

Current garage booking lifecycle, media requirements, Prisma/Neon persistence, and API checkpoints after the booking inspection image update.

Executive Summary

- Customers book services, pay Rovauto handling/platform fee through Cashfree, then nearby garages receive broadcast requests.
- Garage acceptance deducts the configured platform fee from the garage wallet and sends the customer a handover OTP.
- After OTP verification, the garage must upload exactly 5 pickup/modification-area car photos before the booking moves to IN_PROGRESS.
- Before marking delivery, the garage must upload exactly 5 delivery car photos. Both photo sets go to Cloudinary and Neon.
- Customers accept delivery after the garage marks delivered; the booking becomes COMPLETED and appears in service history.

Main Lifecycle

Step	Backend state / side effect
1. Customer checkout	Booking is created with vehicle, services, customer coordinates, price range, and payment row.
2. Cashfree payment	Payment verification moves payable bookings into SEARCHING_GARAGE and starts garage discovery.
3. Garage broadcast	Active verified garages inside GARAGE_BROADCAST_RADIUS_KM receive GarageBroadcastRequest rows and notification/log output.
4. Garage accept	Accepted request sets booking CONFIRMED, assigns garageld, expires other requests, deducts garage accept fee.
5. Handover OTP	Customer receives OTP in app notification metadata/message. Garage must verify it at handover.
6. Pickup photos	Garage uploads exactly 5 images in multipart field images. Backend uploads to Cloudinary and stores PICKUP rows.
7. Work in progress	Booking moves to IN_PROGRESS only after OTP and pickup photo requirements pass.
8. Delivery photos	Garage uploads exactly 5 delivery images. Backend stores DELIVERY rows before setting deliveredAt.
9. Customer acceptance	Customer accepts delivery; booking moves to COMPLETED and service history.

Inspection Image System

The inspection system is separate from garage profile photos. Garage profile photos activate/list a garage; booking inspection photos prove vehicle condition at pickup and delivery.

Step	Backend state / side effect
PICKUP phase	Endpoint: POST /api/v1/garage/requests/:requestId/verify-handover-otp. Payload: otp plus exactly 5 images.
DELIVERY phase	Endpoint: POST /api/v1/garage/requests/:requestId/mark-delivered. Payload: exactly 5 images.
Cloudinary folder	project-x/bookings/inspection-images
Neon table	BookingInspectionImage stores bookingId, garageId, phase, imageUrl, publicId, order, createdAt, updatedAt.
Response shape	Booking list/detail/history and garage request responses include inspectionImages ordered by phase and order.
Validation	Only image files are accepted; each file must be under 10 MB; count must be exactly 5 per phase.

Important Distinction

- Garage activation media: 5 to 10 garage photos through /garages/:garageId/media; stored as GarageImage.
- Booking pickup/delivery media: exactly 5 photos per phase; stored as BookingInspectionImage.
- Complaint images and service media remain separate Cloudinary-backed models.

Key API Routes

Route	Purpose / payload
POST /api/v1/bookings/checkout	Customer creates booking with vehicle, services, location, and wallet usage.
POST /api/v1/payments/verify	Verifies Cashfree payment and triggers garage broadcast for paid bookings.
GET /api/v1/garage/requests	Garage owner sees SENT/ACCEPTED/other request rows for their active garage.
POST /api/v1/garage/requests/:requestId/accept	Accepts request, deducts garage wallet fee, assigns garage, creates handover OTP.
POST /api/v1/garage/requests/:requestId/verify-handover-otp	Multipart: otp and images[5]. Verifies OTP, uploads PICKUP images, moves booking IN_PROGRESS.
POST /api/v1/garage/requests/:requestId/mark-delivered	Multipart: images[5]. Uploads DELIVERY images, sets deliveredAt, notifies customer.
POST /api/v1/bookings/:id/accept-delivery	Customer accepts delivered vehicle; booking moves COMPLETED.
GET /api/v1/bookings/service-history	Returns completed accepted bookings with inspectionImages.
POST /api/v1/garages/:garageId/media	Garage profile photos only: optional thumbnail plus images, total 5 to 10.

Multipart Examples

verify-handover-otp requires form-data fields: otp=123456, images=<5 image files>. mark-delivered requires form-data field: images=<5 image files>.

Prisma / Neon Changes

Route	Purpose / payload
Booking	Now has lifecycle fields in schema: searchExpiresAt, handoverOtpHash, handoverOtpExpiresAt, handoverOtpVerifiedAt, deliveredAt, customerAcceptedAt.
BookingInspectionImage	New table for pickup and delivery images. Unique key: bookingId + phase + order.
BookingPhotoPhase	Enum values: PICKUP, DELIVERY.
Migration	server/prisma/migrations/20260701010000_add_booking_inspection_images/migration.sql

Release Commands

For Neon/production releases:

```
cd server npm run prisma:deploy npm run prisma:generate
```

For local development DBs:

```
cd server npm run prisma:migrate npm run prisma:generate
```

Operational Env Controls

```
GARAGE_BROADCAST_RADIUS_KM=15 GARAGE_SEARCH_TIMEOUT_SECONDS=120 HANDOVER_OTP_TTL_MINUTES=30  
SERVICE_PRICE_RANGE_DELTA=500
```

Notifications And Failure Rules

- Accepted notification includes the garage name and handover OTP for the customer.
- Search timeout expires pending garage requests and tells the customer to try again.
- Invalid or expired OTP blocks pickup photo upload and prevents IN_PROGRESS transition.
- Missing, extra, non-image, or over-10-MB inspection files block pickup/delivery transitions.
- Garage cannot mark delivered until handover OTP is verified and delivery photos pass validation.
- Customer cannot complete booking until the garage has marked delivered and payment is PAID.

Frontend Implementation Notes

- Garage pickup screen should ask for the 6-digit OTP and exactly 5 car photos in the same submit action.
- Garage delivery screen should ask for exactly 5 car photos before enabling mark delivered.
- Use multipart/form-data with repeated images fields; do not send base64 JSON for these endpoints.
- Customer booking detail/history screens can display inspectionImages grouped by PICKUP and DELIVERY.

Generated from current backend state after the booking inspection image update.